UNIVALI

UNIVERSIDADE DO VALE DO ITAJAÍ

CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: SISTEMAS OPERACIONAIS

SIMULADOR DE ESCALONAMENTO MULTIPROCESSADO COM PAGINAÇÃO SOB DEMANDA

GRUPO 5

Thiago Wyse

Matheus Souza

Vitor Hugo

Repositório do código:

https://github.com/thiagowyseoficial/simulador-escalonamento-paginacao.git

Itajaí – SC

Novembro de 2026

SUMÁRIO

1. Introdução.....3
2. O simulador.....3
2.1 Organização e execução.....3
2.2 Modelo de tempo e ordem dos eventos.....4
2.3 Escalonadores e troca de contexto.....5
2.4 Memória paginada: estrutura de dados e justificativa.....6
2.5 Decisões de interpretação.....6
2.6 Validação.....7
3. A carga de trabalho.....7
4. Desenho experimental.....8
5. Resultados.....9
5.1 Tabela resumo — cenário A (LRU, 256 quadros).....9
5.2 Tabela resumo — cenário B (LRU, 256 quadros).....10
5.3 Gráficos.....11
5.4 Diagramas de Gantt (janela 0–200, FCFS, LRU, 256 quadros).....13
5.5 FIFO × LRU (256 quadros).....15
5.6 Variação do número de quadros (FCFS, LRU).....15
6. Análise crítica.....16
6.1 Uma máquina saturada: o custo da troca de contexto domina.....16
6.2 Quatro CPUs: speedup quase ideal no cenário A e superlinear no B.....17
6.3 Escalonadores: a espera média não conta a história toda.....18
6.4 Efeito do quantum no RR.....18
6.5 Injustiça, starvation e efeito comboio.....19
6.6 Thrashing e pressão de memória.....20
6.7 FIFO × LRU.....21
7. Limitações do modelo.....22
8. Conclusões.....22
9. Reprodução.....23
Anexo A — Resumo padronizado de uma execução.....24
Anexo B — Trecho do log de estados (--log).....25
Anexo C — Métricas por CPU (execuções base com 4 CPUs).....26

1. INTRODUÇÃO

Este relatório apresenta o simulador desenvolvido pelo grupo 5, os experimentos feitos com as duas cargas fornecidas pelo professor e a análise dos resultados.

O simulador junta duas partes de um sistema operacional que costumam ser estudadas separadamente: o **escalonamento de CPU** (FCFS, SJF não preemptivo e Round Robin) e a **memória paginada sob demanda** (substituição FIFO ou LRU). Ele simula máquinas com 1 e com 4 CPUs que compartilham uma única fila de prontos e segue as regras da seção 6 do enunciado.

Todos os números do texto vêm dos arquivos gravados em `resultados/`: um JSON de métricas e um CSV de Gantt por execução, além das tabelas resumo.

Para facilitar a leitura, cada execução é citada pelo escalonador, pelo número de CPUs e pelo cenário, por exemplo “SJF, 1 CPU, cenário A”. Salvo indicação em contrário, a execução usa LRU, 256 quadros e custo de troca 1. Os arquivos de cada execução ficam em `resultados/`, numa pasta por cenário; o exemplo acima é o arquivo `sjf_cpus1_lru_256quadros_troca1.json` da pasta `cenario_A_t0`.

O texto está organizado assim: a seção 2 descreve o simulador; as seções 3 e 4, a carga e os experimentos; a seção 5 traz as tabelas e os gráficos; a seção 6 analisa os resultados; e as seções 7 e 8 apresentam as limitações e as conclusões.

2. O SIMULADOR

2.1 Organização e execução

O simulador foi escrito em Python 3 e usa só a biblioteca padrão. A biblioteca `matplotlib` é usada apenas para gerar os gráficos, e o `pytest` apenas para os testes.

Tabela 1 – Arquivos e funções do simulador

Arquivo	Função
simulador.py	O simulador. Lê o JSON, executa, imprime o resumo padronizado (seção 9.1), grava o Gantt 0–200 (9.2) e, com --log, o log de estados (9.3).
experimentos.py	Roda o desenho experimental da seção 11 em paralelo e monta as tabelas resumo.
graficos.py	Gera os gráficos e as figuras de Gantt a partir de resultados/.
tests/test_exemplos.py	16 testes automatizados com os exemplos das seções 5.4, 5.5, 7.1 (FCFS, SJF e RR q=2), 7.2 e 7.3 e com casos de borda.
cargas/	cenario_A_t0.json e cenario_B_espacadas.json do grupo 5.
resultados/, figuras/	Saídas de todas as execuções e figuras do relatório.

Todos os parâmetros são escolhidos na linha de comando, sem editar o código, como pede a seção 8 do enunciado: `--arquivo`, `--cpus`, `--sched` (fcfs, sjf ou rr), `--quantum`, `--subst` (fifo ou lru), `--quadros` (padrão 256), `--custo-troca` (padrão 1), `--fault` (padrão 20) e `--log`. Para controlar o tamanho do log, há ainda `--log-max` e `--log-ate`. Exemplo de execução:

```
python simulador.py --arquivo cargas/cenario_B_espacadas.json --cpus 4 \
    --sched rr --quantum 10 --subst lru --quadros 256 --custo-troca 1 --log
```

Cada execução gera três saídas:

- **Resumo padronizado**, impresso na tela: parâmetros, métricas globais da seção 10.1, tabela por CPU (tempo útil, de troca e ocioso, com porcentagens), page faults e taxa de faults. Há um exemplo completo no Anexo A.
- **Gantt**, gravado em CSV no formato `cpu,inicio,fim,evento`, com uma faixa por CPU.
- **Log de estados** (opcional, com `--log`): a cada evento, registra o evento e o pid, a fila de prontos em ordem, os bloqueados com o instante de retorno, o processo em cada CPU e os quadros ocupados. Há um trecho no Anexo B.

2.2 Modelo de tempo e ordem dos eventos

O tempo é discreto. Em cada instante `t`, o simulador executa cinco passos, sempre nesta ordem:

1. **Entradas na fila:** as chegadas e os retornos de E/S e de page fault entram na fila de prontos, em ordem de pid.

2. **Fim de troca e de quantum:** termina a troca de contexto das CPUs que estavam trocando. No RR, se o quantum acabou e há outro processo pronto, o atual vai para o fim da fila, depois dos que entraram no passo 1. Se não há outro pronto, ele continua sem custo e recebe um novo quantum.
3. **Verificação da referência:** cada processo em execução acessa a página da vez. Se for page fault, ele libera a CPU no mesmo instante, sem consumir a unidade de CPU.
4. **Despachos:** as CPUs livres pegam processos da fila, a de menor índice primeiro. Se o processo despachado é o mesmo que a CPU executou por último, não há troca, e ele já verifica sua referência no mesmo instante. Se der fault, a CPU fica livre de novo e despacha o próximo. É exatamente o que acontece no exemplo 7.2 do enunciado.
5. **Avanço do relógio:** cada CPU soma uma unidade em útil, troca ou ocioso, de modo que útil + troca + ocioso é sempre igual ao makespan.

Quando não há nenhum processo pronto nem CPU ocupada, o relógio pula direto para o próximo evento (uma chegada ou um retorno) e conta esse intervalo como tempo ocioso.

2.3 Escalonadores e troca de contexto

- **FCFS:** usa uma fila simples (`deque`) e atende na ordem de entrada.
- **SJF não preemptivo:** usa um heap ordenado pela parte que falta do burst de CPU atual. Em caso de empate, vence quem entrou primeiro na fila. O SJF usa o campo `cpu` do arquivo e nunca o campo `classe`.
- **RR:** usa a mesma fila do FCFS e um contador de quantum por processo. Um page fault no meio do quantum descarta o que restava dele. Se o burst termina antes do fim do quantum, a CPU é liberada na hora.
- **Troca de contexto (Convenção A):** custa 1 unidade sempre que a CPU recebe um processo diferente do último que executou, inclusive no primeiro despacho. Durante a troca, a CPU fica ocupada e o processo já conta como “executando”; por isso, o tempo de resposta é o instante do despacho menos a chegada, como no exemplo 7.1. A troca não altera a memória.

2.4 Memória paginada: estrutura de dados e justificativa

A memória física é um único `OrderedDict` (um dicionário que guarda a ordem de inserção), com chave `(pid, página)` e no máximo `--quadros` entradas. A própria ordem das chaves é o que as duas políticas de substituição precisam saber:

- **FIFO:** a ordem de inserção já é a ordem de carga. A vítima é a primeira página não protegida.
- **LRU:** a cada acerto, a página vai para o fim do dicionário (`move_to_end`). Assim, o início do dicionário é sempre a página usada há mais tempo, e a vítima é a primeira página não protegida.

Por que essa estrutura. Acertar, carregar e remover uma página custa O(1). A busca da vítima só pula as páginas protegidas, que são no máximo uma por processo bloqueado em fault. Dois auxiliares completam a estrutura: o conjunto `protegidas`, com as páginas que ainda esperam o processo voltar, e o dicionário `pid → páginas`, que libera todos os quadros de um processo quando ele termina. A alternativa simples, percorrer a lista de quadros a cada referência, custaria O(N) por unidade de CPU. Com 34.539 unidades de CPU e até 19 mil faults por execução, a estrutura escolhida mantém cada execução entre 0,2 s e 3 s.

A regra de proteção foi implementada ao pé da letra. No page fault, a página é carregada na hora num quadro (livre ou liberado por substituição) e fica protegida até o processo voltar e repetir a referência, que então é um acerto. As páginas de processos em E/S ou na fila de prontos continuam na memória e podem ser escolhidas como vítima. Quando o processo termina, seus quadros são liberados.

2.5 Decisões de interpretação

Alguns pontos do enunciado admitem mais de uma leitura. As decisões abaixo também estão documentadas no `README.md`:

- **“Sem vítima”:** quando todos os quadros estão protegidos, o processo fica bloqueado 20 unidades e tenta de novo. Entendemos que a nova tentativa refaz o fault: se conseguir um quadro, espera mais 20 unidades pela transferência; se não conseguir, espera mais 20 e tenta outra vez. O fault conta uma única vez, e as novas tentativas aparecem à parte no resumo, como “Retentativas sem vítima”.

- **Taxa de faults:** faults ÷ (acertos + faults), em que cada unidade de CPU executada conta como um acerto.
- **Gantt:** cada troca de contexto aparece como um intervalo próprio no CSV, como no exemplo da seção 9.2 do enunciado.
- **`--fault 0`:** a página é carregada na hora e a execução continua; o fault é contado, mas não bloqueia. Serve apenas para reproduzir nos testes os exemplos “sem memória” do enunciado.

2.6 Validação

Antes de usar as cargas de 1000 processos, resolvemos no papel os exemplos do enunciado e os transformamos em testes automatizados (`python -m pytest -q tests`). Os 16 testes passam:

Tabela 2 – Validação dos exemplos do enunciado

Teste	O que verifica	Valores conferidos
5.4	Processo 17 sem faults	turnaround 19, espera 0
5.5	Processo 17 com faults	4 faults, término em 441, turnaround 99
7.1 FCFS	Troca de contexto = 1	esperas 0/6/8, turnarounds 6/10/11, makespan 13, CPU útil 10, troca 3, Gantt idêntico ao do enunciado
7.1 SJF, RR q=2	Exercícios propostos	SJF: esperas 7/0/2; RR: makespan 16, 6 trocas, Gantt completo
7.2	Faults, fila e CPU ociosa	esperas 0/0/0, turnarounds 12/12/6, 5 faults, útil 5, ocioso 8
7.3 FCFS e SJF	Fila que se forma após faults em paralelo	esperas 0/4/7 (média 3,67) e 5/2/0 (média 2,33)
Bordas	RR com fila vazia sem custo; volta de E/S na mesma CPU sem custo; 4 CPUs com menor índice primeiro; FIFO × LRU com 2 quadros; página protegida nunca é vítima; término libera quadros	—

3. A CARGA DE TRABALHO

Os dois arquivos do grupo 5 têm os mesmos 1000 processos; só os instantes de chegada mudam. A Tabela 3 resume as características medidas diretamente nos arquivos.

Tabela 3 – Caracterização da carga de trabalho

Característica	Valor
Processos	1000: 964 curtos e 36 longos
CPU total da carga	34.539 unidades (curtos 15.872, longos 18.667)
CPU por processo	curto: média 16,5 (2 a 37); longo: média 518,5 (285 a 745)
Bursts de CPU	2.959; média 11,7, mediana 6, máximo 300
Bursts de E/S	média 17,7 unidades (5 a 30); E/S total 34.716
Páginas por processo	média 6,7 (4 a 32); curtos 6,1, longos 24,1; total 6.721
Páginas distintas referenciadas	5.373 (limite inferior de faults = faults compulsórios)
Localidade	em média 63 % das referências de um burst são a páginas distintas
Cenário A	todas as chegadas em t = 0
Cenário B	chegadas de t = 0 a t = 9.076; 3,81 unidades de CPU por unidade de tempo = 95,1 % da capacidade de 4 CPUs

Dois números explicam boa parte dos resultados:

- **Poucos processos longos concentram a CPU.** Os 36 processos longos (3,6 % da carga) usam 54 % do tempo de CPU e têm bursts de até 300 unidades.
- **A memória é muito menor que a carga.** Os processos referenciam 5.373 páginas distintas, mas a memória padrão tem 256 quadros, e nenhuma configuração consegue manter as páginas de muitos processos ao mesmo tempo. Como cada página precisa ser carregada pelo menos uma vez, a menor taxa de faults possível é 5.373 ÷ (34.539 + 5.373) = 13,5 %.

4. DESENHO EXPERIMENTAL

Tabela 4 – Parâmetros do desenho experimental

Dimensão	Valores
Cenários	A (t = 0) e B (espaçadas)
CPUs	1 e 4
Escalonadores	FCFS, SJF, RR com quantum 2, 10 e 50
Memória base	LRU, 256 quadros, custo de troca 1, fault 20
FIFO × LRU	FCFS e RR q=10, com 1 e 4 CPUs, nos dois cenários (8 execuções, 4 por cenário)
Variação de quadros (opcional)	64 e 1024 quadros, FCFS e RR q=10, FIFO e LRU, 1 e 4 CPUs

Por que 20 execuções base, e não 28. O enunciado fala em “RR com 5 quantums” e 28 execuções base, mas a tabela da seção 11 lista só os quantums 2, 10 e 50.

Consultado, o professor confirmou esses três valores. Por isso são 20 execuções base (2 cenários × 2 configurações de CPU × 5 variações de escalonador).

Somando as 8 execuções de FIFO × LRU e as 32 de variação de quadros, são 60 execuções no total. Todas são geradas por um único comando (`experimentos.py`), que leva 7 s numa máquina de 12 núcleos. O speedup e a eficiência sempre comparam execuções com o mesmo escalonador, quantum e memória, como manda a seção 10.1.

5. RESULTADOS

5.1 Tabela resumo — cenário A (LRU, 256 quadros)

Tabela 5 – Cenário A: métricas de desempenho

Escalonador	CPUs	Espera média	Espera máx.	Turnaround médio	Resposta média	Makespan
FCFS	1	10719,38	50945	24239,49	499,5	56577
SJF	1	14133,99	43732	14354,36	7161,29	44981
RR q=2	1	11245,42	58534	24644,37	499,5	64819
RR q=10	1	10755,14	51523	24275,28	499,5	57297
RR q=50	1	10719,53	50928	24239,63	499,5	56592
FCFS	4	2328,68	9454	6185,51	124,5	14339
SJF	4	3510,67	10825	3765,07	1775,25	12083
RR q=2	4	2436,36	11345	6295,70	124,5	16241
RR q=10	4	2337,00	9653	6194,14	124,5	14441
RR q=50	4	2328,68	9454	6185,52	124,5	14341

Tabela 6 – Cenário A: métricas de CPU e memória

Escalonador	CPUs	Speedup	Eficiência	Trocas ctx	Utilização %	Ocioso total	Page faults	Taxa faults %
FCFS	1	—	—	21988	99,91	50	19031	35,53
SJF	1	—	—	9998	99,01	444	7056	16,96
RR q=2	1	—	—	30280	100,0	0	18845	35,30
RR q=10	1	—	—	22748	99,98	10	18995	35,48
RR q=50	1	—	—	21992	99,89	61	19031	35,53
FCFS	4	3,946	0,986	21994	98,57	823	19044	35,54
SJF	4	3,723	0,931	11624	95,51	2169	8676	20,08
RR q=2	4	3,991	0,998	30161	99,59	264	18949	35,43
RR q=10	4	3,968	0,992	22749	99,18	476	19022	35,51
RR q=50	4	3,946	0,987	22000	98,56	825	19044	35,54

5.2 Tabela resumo — cenário B (LRU, 256 quadros)

Tabela 7 – Cenário B: métricas de desempenho

Escalonador	CPUs	Espera média	Espera máx.	Turnaround médio	Resposta média	Makespan
FCFS	1	10145,25	49603	18969,84	488,91	55790
SJF	1	9060,23	40460	9269,99	3829,36	44499
RR q=2	1	10860,57	58048	19892,56	488,77	64650
RR q=10	1	10185,31	50145	19010,39	488,91	56572
RR q=50	1	10145,54	49584	18970,12	488,91	55810
FCFS	4	487,45	5003	748,85	40,62	11908
SJF	4	120,72	7292	306,36	15,24	11337
RR q=2	4	950,12	9336	1296,68	40,26	15446
RR q=10	4	481,59	5451	755,74	34,82	12234
RR q=50	4	489,88	5122	756,50	39,06	11887

Tabela 8 – Cenário B: métricas de CPU e memória

Escalonador	CPUs	Speedup	Eficiência	Trocas ctx	Utilização %	Ocioso total	Page faults	Taxa faults %
FCFS	1	—	—	21227	99,96	24	18268	34,59
SJF	1	—	—	9492	98,95	468	6551	15,94
RR q=2	1	—	—	30111	100,0	0	18381	34,73
RR q=10	1	—	—	22006	99,95	27	18254	34,58
RR q=50	1	—	—	21231	99,93	40	18268	34,59
FCFS	4	4,685	1,171	11966	97,63	1127	9009	20,69
SJF	4	3,925	0,981	8347	94,57	2462	5402	13,52
RR q=2	4	4,186	1,046	26708	99,13	537	12530	26,62
RR q=10	4	4,624	1,156	13459	98,08	938	9572	21,70
RR q=50	4	4,695	1,174	12265	98,44	744	9255	21,13

As Tabelas 5 a 8 mostram os totais de cada execução. Os valores de cada CPU (trocas, utilização e tempo ocioso) nas execuções com 4 CPUs estão no Anexo C e no JSON de cada execução, em `resultados/<cenário>/`.

5.3 Gráficos

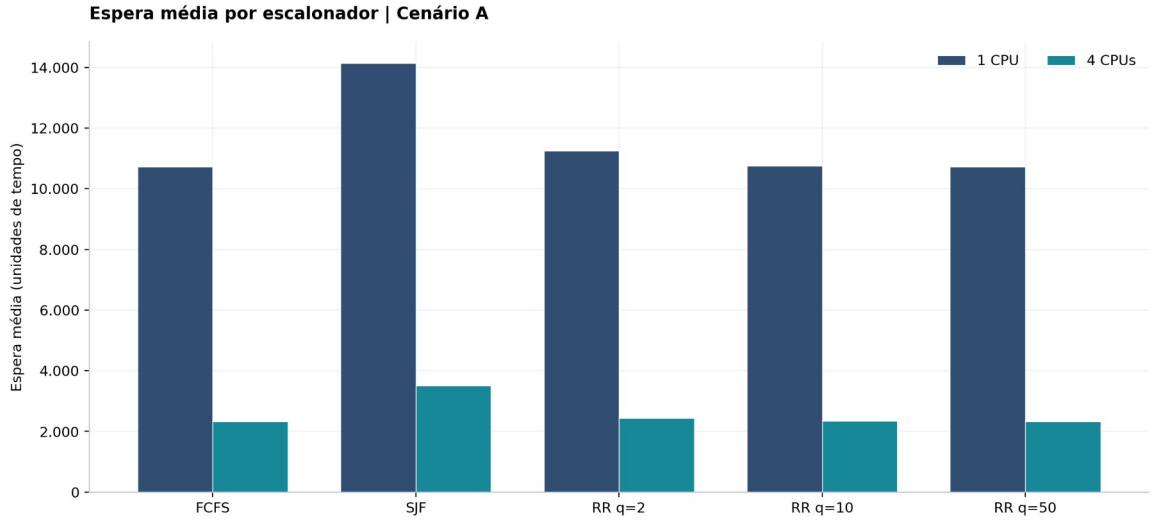


Figura 1 – Espera média por escalonador, cenário A.

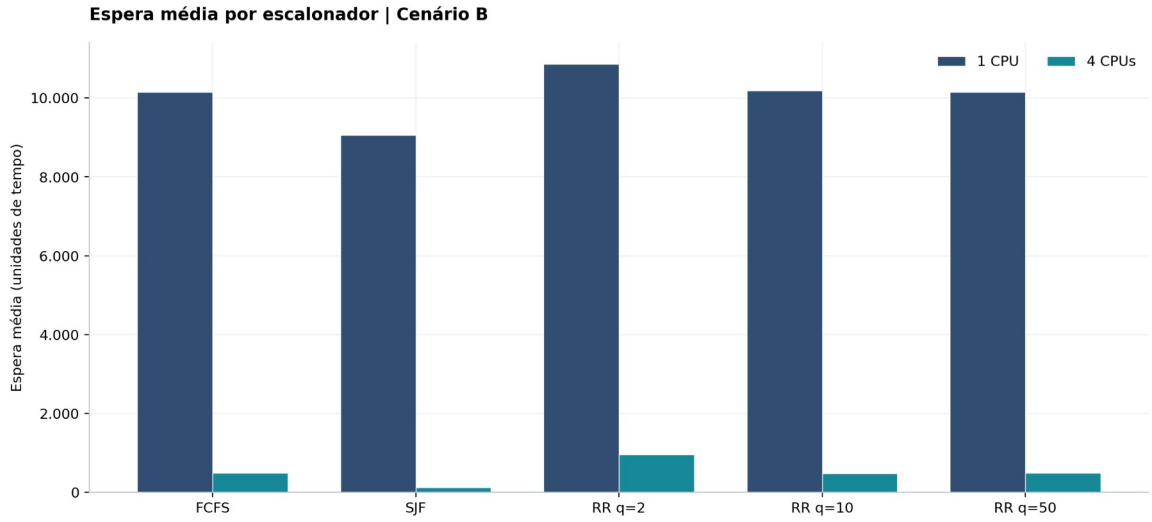


Figura 2 – Espera média por escalonador, cenário B.

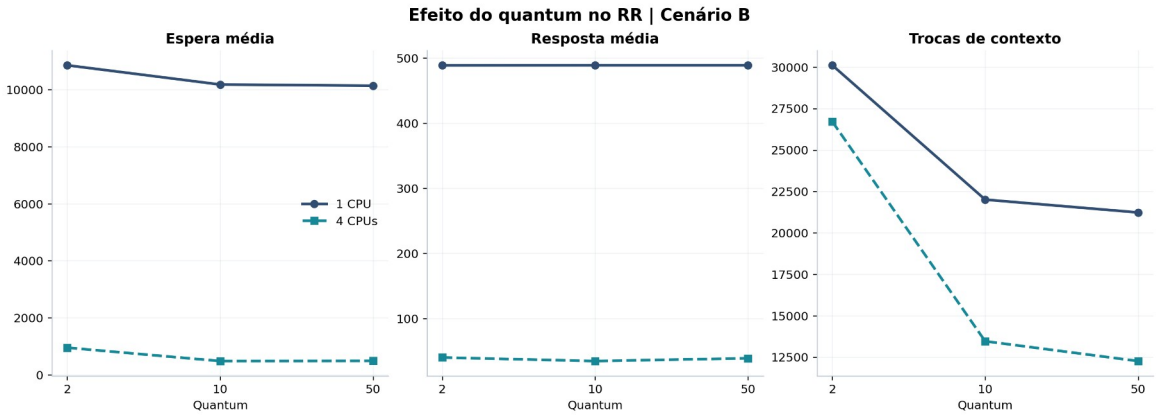


Figura 3 – Efeito do quantum no RR (espera, resposta e trocas de contexto), cenário B.

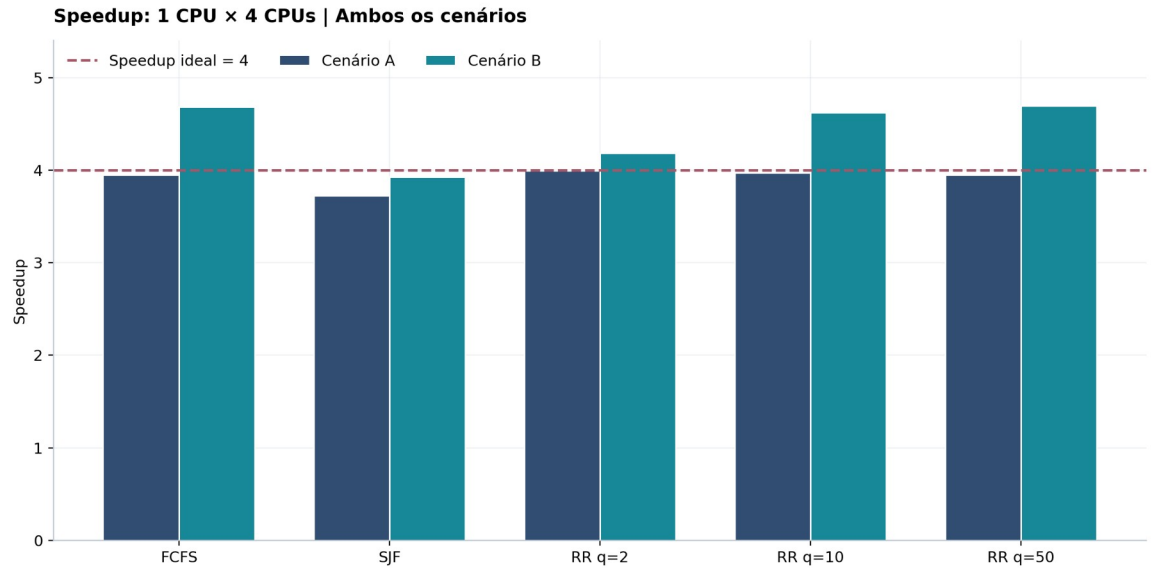


Figura 4 – Speedup por escalonador nos dois cenários (linha pontilhada = ideal 4).

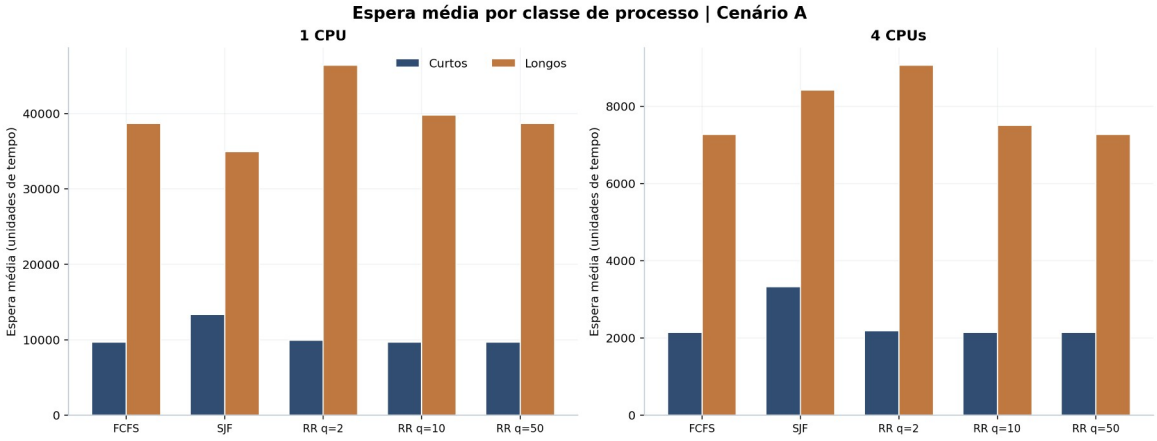


Figura 5 – Espera média de processos curtos e longos por escalonador, cenário A.

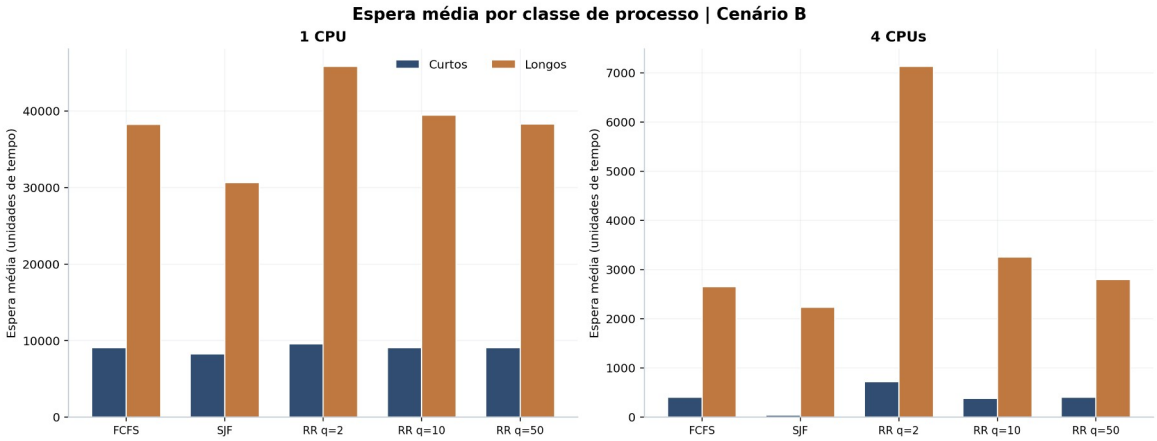


Figura 6 – Espera média de processos curtos e longos por escalonador, cenário B.

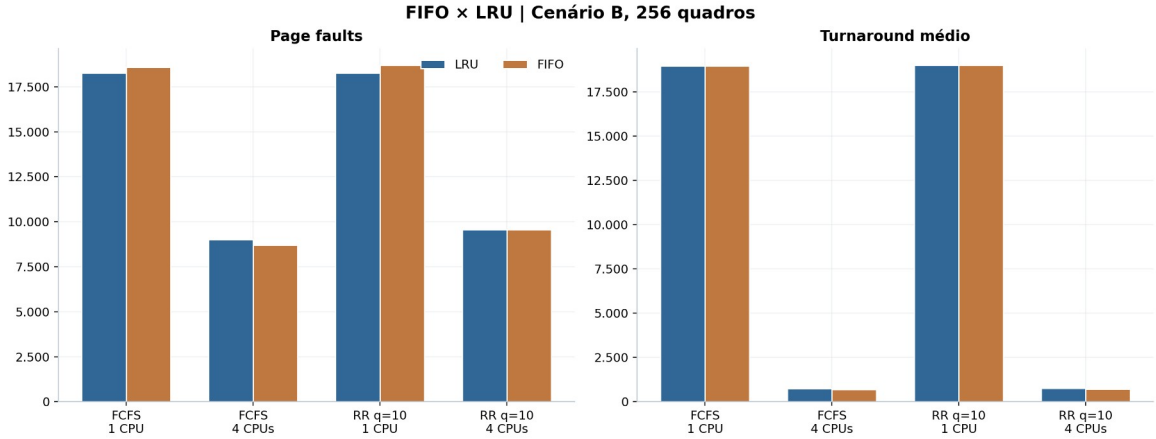


Figura 7 – Page faults e turnaround com FIFO e LRU, cenário B.

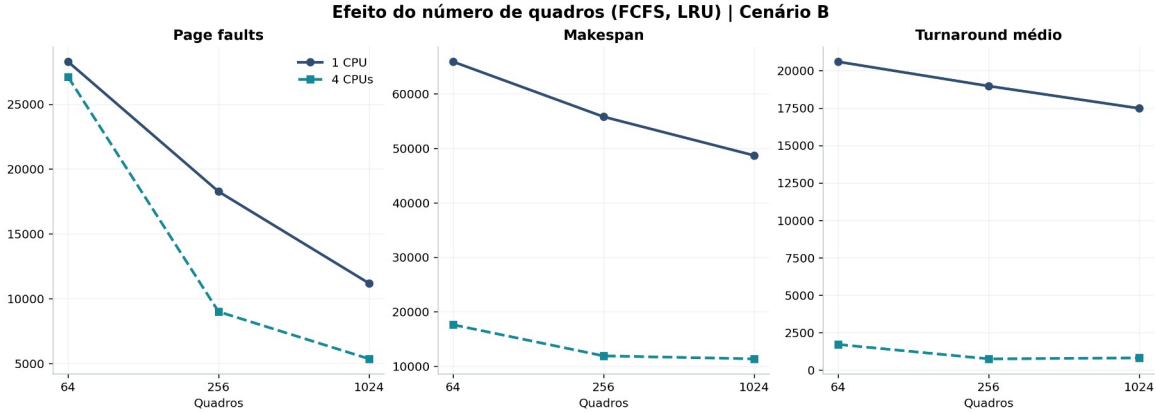


Figura 8 – Efeito do número de quadros (FCFS, LRU), cenário B.

Os gráficos equivalentes do outro cenário estão na pasta `figuras/`.

5.4 Diagramas de Gantt (janela 0–200, FCFS, LRU, 256 quadros)

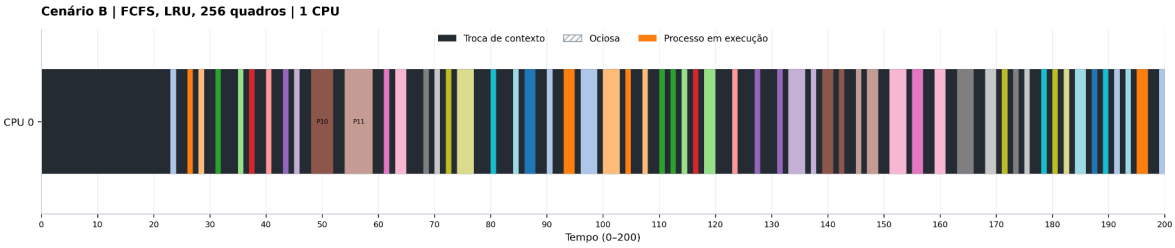


Figura 9 – Gantt do cenário B com 1 CPU (fcfs_cpus1_lru_256quadros_troca1).

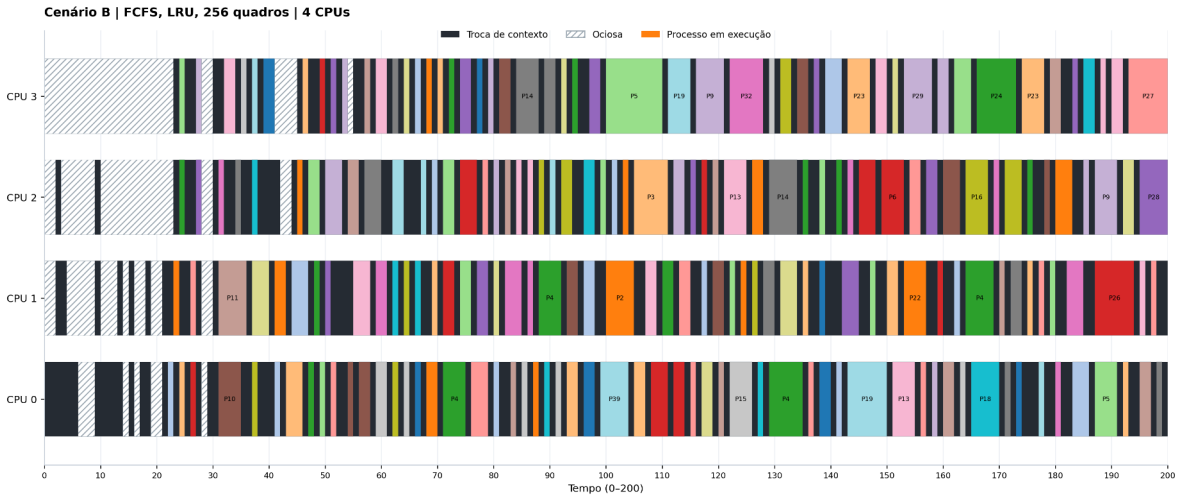


Figura 10 – Gantt do cenário B com 4 CPUs (fcfs_cpus4_lru_256quadros_troca1).

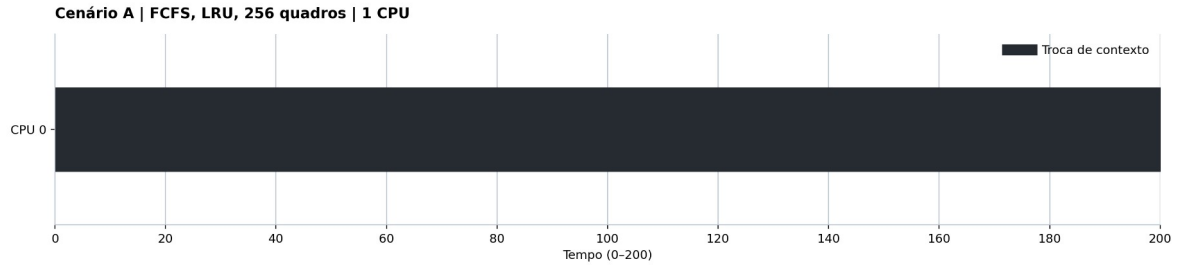


Figura 11 – Gantt do cenário A com 1 CPU (fcfs_cpus1_lru_256quadros_troca1).

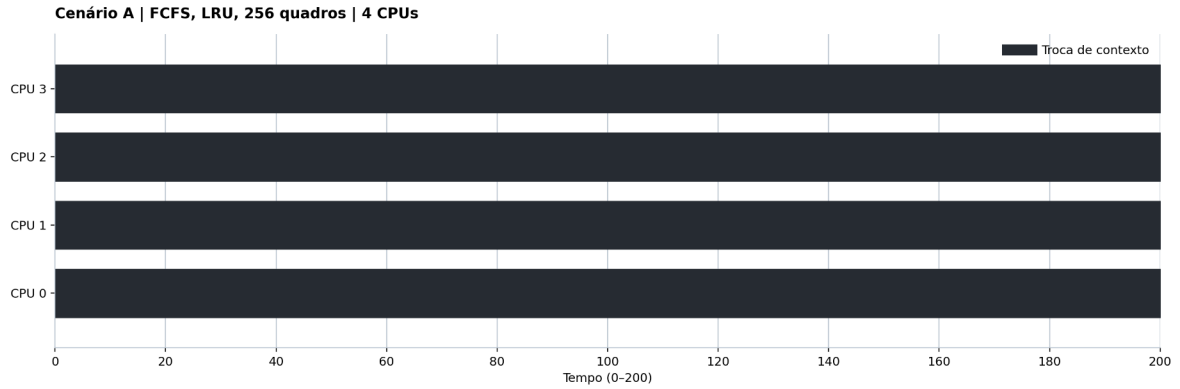


Figura 12 – Gantt do cenário A com 4 CPUs (fcfs_cpus4_lru_256quadros_troca1).

Cenário B. Nos primeiros 20 instantes as CPUs ficam ociosas (hachurado), porque os primeiros processos estão esperando seus page faults iniciais. Depois, o Gantt alterna trocas de contexto (preto) e bursts curtos.

Cenário A. As duas figuras ficam inteiramente pretas. Isso é o comportamento correto do modelo, não um erro. Os 1000 processos chegam em t = 0 com a memória vazia. Cada despacho custa 1 unidade de troca, e a primeira referência de

cada processo é um page fault, que libera a CPU sem usar CPU. Assim, as CPUs fazem 1000 trocas seguidas (250 por CPU com 4 CPUs).

Os processos que voltam do fault em t = 21 vão para o fim de uma fila com centenas de processos. A primeira unidade útil de CPU só acontece em t = 251 com 4 CPUs (P1 na CPU 0, visível com `--gantt-fim 600`) e em t = 1001 com 1 CPU. O log de estados (Anexo B) mostra essa sequência evento a evento.

5.5 FIFO × LRU (256 quadros)

Tabela 9 – Comparação FIFO × LRU, 256 quadros

Cenário	Escalonador	CPUs	Faults LRU	Faults FIFO	Makespan LRU	Makespan FIFO	Turnaround LRU	Turnaround FIFO
A	FCFS	1	19031	19320	56577	56838	24239,49	24247,39
A	FCFS	4	19044	19365	14339	14404	6185,51	6181,70
A	RR q=10	1	18995	19452	57297	57723	24275,28	24284,63
A	RR q=10	4	19022	19455	14441	14555	6194,14	6189,03
B	FCFS	1	18268	18589	55790	56130	18969,84	18983,05
B	FCFS	4	9009	8697	11908	11881	748,85	689,62
B	RR q=10	1	18254	18701	56572	56959	19010,39	19018,08
B	RR q=10	4	9572	9557	12234	12214	755,74	725,05

5.6 Variação do número de quadros (FCFS, LRU)

Tabela 10 – Variação do número de quadros (FCFS, LRU)

Cenário	CPUs	Quadros	Page faults	Taxa %	Makespan	Turnaround médio	Espera média	Utilização %	Ocioso total	Trocas ctx
A	1	64	28493	45,20	66103	25034,62	3614,62	99,82	117	31447
A	1	256	19031	35,53	56577	24239,49	10719,38	99,91	50	21988
A	1	1024	12995	27,34	50493	25661,13	25316,03	100,0	0	15954
A	4	64	28612	45,31	17974	6294,99	414,83	91,94	5797	31560
A	4	256	19044	35,54	14339	6185,51	2328,68	98,57	823	21994
A	4	1024	12994	27,34	12768	6420,43	6075,34	98,86	580	15953
B	1	64	28276	45,01	65889	20593,37	3228,72	99,82	119	31231
B	1	256	18268	34,59	55790	18969,84	10145,25	99,96	24	21227
B	1	1024	11200	24,49	48698	17481,57	17174,15	100,0	0	14159
B	4	64	27103	43,97	17626	1709,05	256,78	91,62	5911	30054
B	4	256	9009	20,69	11908	748,85	487,45	97,63	1127	11966
B	4	1024	5373	13,46	11366	808,35	623,32	94,27	2606	8319

A tabela completa, com RR q=10 e FIFO, está em `resultados/quadros_<cenário>.md`.

6. ANÁLISE CRÍTICA

Resumo da análise. Os resultados se explicam por quatro fatos, detalhados nas seções a seguir:

- a memória é pequena para a carga, e por isso page faults e trocas de contexto ocupam boa parte do tempo da CPU (seções 6.1 e 6.6);
- com 4 CPUs a execução fica quase 4 vezes mais rápida no cenário A e mais de 4 vezes no cenário B, porque a fila de prontos fica curta (seção 6.2);
- o SJF tem o menor turnaround, mas faz os processos longos esperarem muito (seções 6.3 e 6.5);
- o quantum só faz diferença quando é muito pequeno (q=2), e FIFO e LRU quase empatam (seções 6.4 e 6.7).

6.1 Uma máquina saturada: o custo da troca de contexto domina

Com 1 CPU, as duas cargas mantêm a CPU ocupada o tempo todo: a utilização fica entre 98,95 % e 100 % em todos os escalonadores (Tabelas 6 e 8). Mas ocupada não quer dizer produtiva.

Em FCFS, 1 CPU, cenário A, o makespan de 56.577 unidades se divide em 34.539 de CPU útil (a carga inteira), 21.988 de troca de contexto e 50 ociosas. Ou seja, **39 % do tempo da CPU é gasto trocando de contexto**. No cenário B acontece o mesmo: 21.227 unidades de troca num makespan de 55.790.

Por que isso acontece. Troca de contexto e page fault se alimentam um do outro:

- com 256 quadros para 5.373 páginas, quase todo acesso a uma página nova é fault;
- cada fault tira o processo da CPU, e o próximo despacho é de outro processo, o que custa mais uma troca (são 21.988 trocas para 19.031 faults, quase uma por fault);
- o processo que deu fault volta para o fim de uma fila de até 1000 processos e, quando chega de novo à CPU, suas outras páginas já foram substituídas.

Por isso a taxa de faults (35,5 %) é 2,6 vezes maior que o mínimo possível (13,5 %).

O SJF escapa em parte desse ciclo. Em SJF, 1 CPU, cenário A, são 9.998 trocas e 7.056 faults (16,96 %), perto do mínimo. O processo que volta de um fault continua

com o mesmo burst curto e é escolhido de novo logo, enquanto suas páginas ainda estão na memória. O resultado é o menor makespan com 1 CPU (44.981, 20 % menor que os 56.577 do FCFS) e um turnaround médio de 14.354, contra 24.239 do FCFS.

6.2 Quatro CPUs: speedup quase ideal no cenário A e superlinear no B

Cenário A: quase o ideal. O speedup fica entre 3,72 (SJF) e 3,99 (RR q=2), com eficiência de 0,93 a 1,00 (Tabela 6 e Figura 4). O ganho é quase ideal porque a máquina de 1 CPU já estava 100 % ocupada e as 4 CPUs fazem exatamente o mesmo trabalho: o mesmo número de trocas (21.988 contra 21.994 no FCFS) e de faults (19.031 contra 19.044).

O SJF fica um pouco abaixo (eficiência 0,931). No fim da execução só restam os 36 processos longos, que não bastam para ocupar 4 CPUs, e sobram 2.169 unidades ociosas.

Cenário B: mais que o ideal. O speedup chega a 4,685 no FCFS e a 4,695 no RR q=50, ou seja, eficiência maior que 1. Isso não significa que cada CPU ficou mais rápida: com 4 CPUs há **menos trabalho total** a fazer, como mostra a Tabela 11 (FCFS, cenário B):

Tabela 11 – Decomposição do makespan por CPU

	CPU útil	Troca	Ocioso	Soma	Makespan
1 CPU	34539	21227	24	55790	55790
4 CPUs	34539	11966	1127	47632	47632 ÷ 4 = 11908

Com 4 CPUs há 9.261 trocas a menos e 9.259 faults a menos (9.009 contra 18.268). As duas reduções têm a mesma causa: a fila de prontos é bem mais curta (no máximo 107 processos, contra 372 com 1 CPU). Com a fila curta:

- um processo que volta de E/S muitas vezes encontra livre a mesma CPU em que estava, e aí não paga troca (regra 6.4 do enunciado);
- poucos processos disputam a memória ao mesmo tempo, então as páginas de quem volta ainda estão lá.

O SJF não tem esse ganho (speedup 3,925) porque, com 1 CPU, ele já evitava a maior parte dos faults (6.551) e das trocas (9.492). Não havia desperdício para eliminar.

6.3 Escalonadores: a espera média não conta a história toda

Cenário B com 4 CPUs: o resultado clássico. O SJF tem a menor espera média (120,72, contra 487,45 do FCFS e 481,59 do RR q=10), a menor resposta média (15,24) e o menor turnaround (306,36, contra 748,85 do FCFS), como mostram a Tabela 7 e a Figura 2. Com 964 processos curtos e 36 longos, atender primeiro o burst menor reduz a espera média e, aqui, ainda evita faults (5.402, quase só os inevitáveis).

Cenário A com 1 CPU: um resultado que parece contraditório. O SJF tem a **maior** espera média (14.134, contra 10.719 do FCFS) e, ao mesmo tempo, o **menor** turnaround (14.354, contra 24.239). A explicação está no que a espera mede: só o tempo na fila de prontos. No FCFS, os processos passam a maior parte da vida bloqueados em faults, e esse tempo não conta como espera. No SJF, com um terço dos faults, esse tempo vira espera na fila.

Por isso, a métrica que melhor mostra o que o usuário sente é o turnaround, e nela o SJF vence nas quatro configurações. A mesma armadilha aparece na variação de quadros (seção 6.6): com 64 quadros a espera média cai para um terço, mas o sistema fica muito pior.

RR com quantum 10 ou 50 quase não difere do FCFS. Em espera, turnaround e makespan, as diferenças ficam abaixo de 3 % (por exemplo, makespan de 56.577 no FCFS, 57.297 com q=10 e 56.592 com q=50, no cenário A com 1 CPU). A mediana dos bursts é de 6 unidades e os faults já interrompem os processos, então o quantum quase nunca chega ao fim: são só 5 preempções com q=50 e 794 com q=10, em 21 mil despachos. O RR só se diferencia do FCFS quando os bursts são maiores que o quantum, o que é raro nesta carga.

6.4 Efeito do quantum no RR

Só q=2 muda o comportamento, e para pior (Figura 3). No cenário B com 4 CPUs, ao passar de q=10 para q=2:

- as trocas de contexto dobram, de 13.459 para 26.708;
- o makespan sobe 26 %, de 12.234 para 15.446;
- a espera média quase dobra, de 481,59 para 950,12, e o turnaround vai de 755,74 para 1.296,68.

O motivo são as preempções: 11.228 com q=2, contra 932 com q=10 e 51 com q=50. Cada preempção custa uma troca, e com q=2 o processo paga uma unidade de troca a cada duas de trabalho. Nos outros três casos (cenário A, e cenário B com 1 CPU) o efeito é menor, mas vai na mesma direção: as trocas sobem de 33 % a 37 % e o makespan de 12 % a 14 %.

O quantum pequeno também piora a memória: no cenário B com 4 CPUs, os faults sobem de 9.572 (21,7 %) com q=10 para 12.530 (26,6 %) com q=2. Cada preempção manda o processo para o fim da fila, e suas páginas ficam mais tempo expostas à substituição.

O tempo de resposta quase não depende do quantum: 40,26, 34,82 e 39,06 (q=2, 10 e 50) no cenário B com 4 CPUs, e 499,5 em todos os quantums no cenário A com 1 CPU. No cenário A isso é uma conta simples: com 1 CPU, o k-ésimo processo é despachado em t = k − 1, e a média de 0 a 999 é 499,5. Como todos dão fault no primeiro acesso, antes de qualquer quantum acabar, o quantum não interfere.

Recomendação: para esta carga, q=10 é a escolha mais equilibrada. Ele mantém a resposta rápida quando há bursts longos, evita a sobrecarga de q=2 e não vira um FCFS, como acontece com q=50.

6.5 Injustiça, starvation e efeito comboio

Injustiça contra os longos no SJF. As Figuras 5 e 6 separam os processos curtos dos longos. No cenário B com 4 CPUs, o SJF dá aos curtos uma espera média de 42 unidades e aos longos de 2.230, 53 vezes mais. No FCFS, os valores são 407 e 2.650, uma diferença de só 6,5 vezes. Ou seja, a espera média baixa do SJF é paga pelos 36 processos longos: a espera máxima do SJF (7.292, pid 325) é maior que a do FCFS (5.003, pid 272), embora a espera média seja quatro vezes menor.

Starvation na prática. Em SJF, 1 CPU, cenário A, o processo 325 (longo, com 580 unidades de CPU, maior burst de 300 e 32 páginas) só é despachado pela primeira vez em t = 42.615, quando o makespan é 44.981. Ele esperou 95 % da execução para começar, porque sempre havia um burst menor na fila. Nessa execução, a resposta média dos longos é 29.082, contra 437 no FCFS.

A espera só não é infinita porque a carga é finita e as chegadas param. Num sistema em que processos curtos continuassem chegando, o processo 325 nunca executaria. Ele é o processo de maior espera nas quatro execuções do SJF.

Efeito comboio no FCFS. No cenário B com 4 CPUs, os processos curtos esperam em média 407 unidades no FCFS e 42 no SJF, quase 10 vezes mais, com a mesma carga e a mesma máquina. A causa são os bursts de até 300 unidades dos processos longos: enquanto um longo ocupa a CPU, os curtos que chegaram depois ficam enfileirados atrás dele. O RR com q=10 quebra esse comboio, mas aqui o ganho é pequeno (espera dos curtos de 378, contra 407) porque os faults já interrompem os longos antes do fim do quantum.

6.6 Thrashing e pressão de memória

Com 1000 processos e 256 quadros, o cenário A está em **thrashing**: os processos passam mais tempo esperando páginas do que executando. Os sinais, em FCFS, 1 CPU, cenário A:

- taxa de faults de 35,5 %, contra o mínimo de 13,5 %, com 17.363 substituições de página;
- 638.733 retentativas “sem vítima”: todos os quadros estão ocupados por páginas protegidas, e quem dá fault não consegue nem um quadro e volta a esperar 20 unidades (com 4 CPUs são 160.662; no cenário B com 4 CPUs, nenhuma, porque a fila é curta);
- turnaround médio de 24.239 unidades, para processos que precisam em média de só 69 unidades de CPU e E/S.

Menos memória, CPU mais ociosa. A variação de quadros (Tabela 10 e Figura 8) confirma o diagnóstico. Com 64 quadros no cenário B com 4 CPUs, a taxa de faults vai a 44 %, as CPUs ficam ociosas 5.911 unidades (utilização de 91,6 %) mesmo com 1000 processos para executar, e o turnaround médio mais que dobra (1.709 contra 749). É o sinal clássico de thrashing: com mais pressão sobre a memória, a utilização da CPU **cai**, porque os processos estão todos bloqueados em faults.

Mais memória resolve o cenário B. Com 1024 quadros, os faults caem exatamente ao mínimo (5.373, 13,46 %), não há nenhuma substituição, e FIFO e LRU dão resultados idênticos.

No cenário A, nem 1024 quadros bastam. Os faults ficam em 12.995 (27 %), bem acima dos 5.373 inevitáveis. Com 1000 processos circulando na fila, o total de páginas usadas entre duas passagens de um processo pela CPU é maior que a memória. Nesse padrão cíclico, o LRU se comporta como o FIFO e descarta justamente a página que será usada em seguida.

O SJF é o único escalonador sem nenhuma retentativa “sem vítima” nas quatro execuções. Como o processo que volta de um fault é escalonado logo, ele libera rápido a proteção da sua página, e o número de páginas protegidas fica abaixo de 256.

6.7 FIFO × LRU

Nas oito comparações (Tabela 9 e Figura 7), FIFO e LRU diferem menos de 4 % em faults e menos de 1 % em makespan. O LRU tem menos faults em seis delas: as quatro com 1 CPU e as duas do cenário A com 4 CPUs (por exemplo, 19.031 contra 19.320 em FCFS, 1 CPU, cenário A). No cenário B com 4 CPUs, o FIFO tem menos faults (8.697 contra 9.009 no FCFS; 9.557 contra 9.572 no RR q=10) e também um turnaround menor (689,62 contra 748,85 no FCFS, 8 % a menos; 725,05 contra 755,74 no RR q=10, 4 % a menos). Nas outras seis comparações, o turnaround difere menos de 1 %.

Por que a diferença é pequena. Há duas razões:

- a maior parte dos faults é inevitável (5.373 dos 9.009 em FCFS, 4 CPUs, cenário B), e nenhuma política evita o primeiro acesso a uma página;
- a regra de proteção tira da disputa as páginas mais recentes. Sobram para vítima as páginas de processos na fila ou em E/S, e para elas “carregada há mais tempo” (FIFO) e “usada há mais tempo” (LRU) quase coincidem, porque a maioria dos processos executa poucas unidades entre dois bloqueios.

O caso em que o FIFO vence mostra que o LRU não é ótimo. Com cerca de 100 processos intercalados, uma página usada recentemente não tende a ser usada de novo logo, e o LRU perde a vantagem. O FIFO, por ignorar os acertos, acaba retirando páginas de processos que saíram da CPU há mais tempo.

7. LIMITAÇÕES DO MODELO

- **O SJF conhece o futuro.** Ele usa o valor exato do campo `cpu` do próximo burst (ou da parte que falta). Num sistema real esse valor é desconhecido e precisa ser estimado, por exemplo pela média exponencial dos bursts anteriores. Por isso, os resultados do SJF são o melhor caso possível.
- **Page fault de custo fixo e sem fila de disco.** Os processos bloqueados esperam em paralelo, 20 unidades cada. Num sistema real, 1000 faults simultâneos formariam fila no disco, e o thrashing do cenário A seria ainda pior.
- **Troca de contexto de custo fixo e sem efeito na memória.** Não há cache nem TLB. A única “afinidade” é não pagar troca quando o mesmo processo volta para a mesma CPU.
- **Memória simplificada.** Não há endereços, offsets nem páginas compartilhadas; cada quadro guarda apenas a identidade (pid, página).
- **A regra de retentativa é uma interpretação** (seção 2.5). Na outra leitura possível, a retentativa que consegue um quadro não espera mais 20 unidades. Isso reduziria o turnaround no cenário A, mas não mudaria as conclusões, porque o número de faults e de retentativas seria o mesmo.
- **Referências e chegadas são sintéticas**, geradas pelo professor. Programas reais costumam ter mais localidade que esta carga, em que 63 % dos acessos de um burst são a páginas distintas. Mais localidade favoreceria o LRU.
- **O log de estados fica muito grande:** no cenário A passa de 700 mil linhas. Por isso, cada linha lista no máximo 20 pids por conjunto, com o total entre parênteses (`--log-max`), e `--log-ate T` grava o log só até o instante T, que é o modo indicado para a defesa.

8. CONCLUSÕES

1. Com 1 CPU, a troca de contexto ocupa 38–39 % do tempo no FCFS e no RR, porque page faults e trocas se alimentam um do outro.

2. Com 4 CPUs, o speedup é de 3,7 a 4,0 no cenário A e passa de 4 no cenário B (até 4,7), porque a fila curta elimina metade das trocas e dos faults.

3. O SJF tem o menor turnaround e o menor número de faults em todas as configurações, mas deixa os 36 processos longos esperando muito: o pid 325 esperou 95 % da execução para começar.
4. O FCFS sofre efeito comboio: no cenário B com 4 CPUs, os processos curtos esperam 10 vezes mais do que no SJF.
5. No RR, os quantums 10 e 50 dão resultados quase iguais aos do FCFS nesta carga. Já q=2 aumenta as trocas em 33 % a 98 % e o makespan em 12 % a 26 % em relação a q=10.
6. Com 256 quadros para 5.373 páginas, o cenário A entra em thrashing: 35 % de faults e 638 mil retentativas sem vítima. Com 64 quadros, a utilização da CPU cai para 92 % mesmo com 1000 processos para executar. Com 1024 quadros, o cenário B chega ao mínimo possível de faults.
7. FIFO e LRU diferem menos de 4 % em faults, porque a maioria dos faults é inevitável e a regra de proteção apaga quase toda a diferença entre as duas políticas.

9. REPRODUÇÃO

No Linux ou macOS:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q tests
.venv/bin/python experimentos.py --cargas cargas/cenario_A_t0.json
cargas/cenario_B_espacadas.json \
    --quantums 2 10 50 --quadros-extra 64 1024 --saida resultados
.venv/bin/python graficos.py --resultados resultados --saida figuras
```

No Windows (PowerShell), os mesmos passos são:

```
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python -m pytest -q tests
.venv\Scripts\python experimentos.py --cargas cargas/cenario_A_t0.json
cargas/cenario_B_espacadas.json `
    --quantums 2 10 50 --quadros-extra 64 1024 --saida resultados
.venv\Scripts\python graficos.py --resultados resultados --saida figuras
```

O simulador é determinístico: as mesmas entradas geram exatamente os mesmos arquivos em `resultados/`.

ANEXO A — RESUMO PADRONIZADO DE UMA EXECUÇÃO

```
python simulador.py --arquivo cargas/cenario_B_espacadas.json --cpus 4 --sched sjf
--subst lru --quadros 256 --custo-troca 1
```

```
================================================================================
RESUMO DA EXECUÇÃO
================================================================================
Parâmetros:
  arquivo=cargas/cenario_B_espacadas.json  cpus=4  sched=sjf  subst=lru  quadros=256
custo_troca=1  fault=20
  processos=1000  cpu_total_carga=34539  es_total_carga=34716
--------------------------------------------------------------------------------
Métricas globais:
  Espera média        : 120.72
  Espera máxima       : 7292 (pid 325)
  Turnaround médio    : 306.36
  Resposta média      : 15.24
  Makespan            : 11337
  Trocas de contexto  : 8347 (preempções por quantum: 0)
  Utilização média    : 94.57%
  Tamanho máx. da fila: 35
  Classe curto : n=964  espera média=41.97  espera máx=300  turnaround médio=194.65  resposta
média=7.13
  Classe longo : n=36  espera média=2229.50  espera máx=7292  turnaround médio=3297.72
resposta média=232.44
--------------------------------------------------------------------------------
Por CPU:
  CPU     útil    troca   ocioso    útil%   troca%  ocioso%  utiliz.  trocas
    0     8698     2438      201   76.72%   21.50%    1.77%   98.23%    2438
    1     8568     2124      645   75.58%   18.74%    5.69%   94.31%    2124
    2     8562     2147      628   75.52%   18.94%    5.54%   94.46%    2147
    3     8711     1638      988   76.84%   14.45%    8.71%   91.29%    1638
--------------------------------------------------------------------------------
Memória:
  Page faults totais  : 5402
  Referências totais  : 39941 (acertos + faults)
  Taxa de faults      : 13.52% (faults / referências)
  Substituições       : 34   Retentativas sem vítima: 0
================================================================================
```

ANEXO B — TRECHO DO LOG DE ESTADOS (--LOG)

Primeiros instantes da mesma execução. Cada linha traz o evento, a fila de prontos em ordem, os bloqueados com instante de retorno, o processo em cada CPU e os quadros ocupados.

```
t=0 | CHEGADA pid=1 | prontos=[1] | bloqueados={} | cpus=['-', '-', '-', '-'] | quadros=0/256
t=0 | DESPACHO pid=1 cpu=0 troca_ate=1 | prontos=[] | bloqueados={} | cpus=['P1(troca)', '-',
'-', '-'] | quadros=0/256
t=1 | CHEGADA pid=2 | prontos=[2] | bloqueados={} | cpus=['P1(troca)', '-', '-', '-'] |
quadros=0/256
t=1 | FAULT pid=1 cpu=0 pag=6 retorno=21 | prontos=[2] | bloqueados={1:21} | cpus=['-', '-',
'-', '-'] | quadros=1/256
t=1 | DESPACHO pid=2 cpu=0 troca_ate=2 | prontos=[] | bloqueados={1:21} | cpus=['P2(troca)',
'-', '-', '-'] | quadros=1/256
t=2 | CHEGADA pid=3 | prontos=[3] | bloqueados={1:21} | cpus=['P2(troca)', '-', '-', '-'] |
quadros=1/256
t=2 | CHEGADA pid=4 | prontos=[4, 3] | bloqueados={1:21} | cpus=['P2(troca)', '-', '-', '-'] |
quadros=1/256
t=2 | CHEGADA pid=5 | prontos=[5, 4, 3] | bloqueados={1:21} | cpus=['P2(troca)', '-', '-',
'-'] | quadros=1/256
t=2 | FAULT pid=2 cpu=0 pag=4 retorno=22 | prontos=[5, 4, 3] | bloqueados={1:21, 2:22} |
cpus=['-', '-', '-', '-'] | quadros=2/256
t=2 | DESPACHO pid=5 cpu=0 troca_ate=3 | prontos=[4, 3] | bloqueados={1:21, 2:22} |
cpus=['P5(troca)', '-', '-', '-'] | quadros=2/256
t=2 | DESPACHO pid=4 cpu=1 troca_ate=3 | prontos=[3] | bloqueados={1:21, 2:22} |
cpus=['P5(troca)', 'P4(troca)', '-', '-'] | quadros=2/256
t=2 | DESPACHO pid=3 cpu=2 troca_ate=3 | prontos=[] | bloqueados={1:21, 2:22} |
cpus=['P5(troca)', 'P4(troca)', 'P3(troca)', '-'] | quadros=2/256
```

No SJF a fila é mostrada em ordem de despacho (menor burst primeiro): em t=2 o processo 5 tem o menor burst e sai antes do 4 e do 3, embora tenha chegado depois.

ANEXO C — MÉTRICAS POR CPU (EXECUÇÕES BASE COM 4 CPUS)

Utilização = (útil + troca) ÷ makespan; ocioso = makespan − útil − troca. Em cada linha, útil + troca + ocioso = makespan da execução.

Cenário A (LRU, 256 quadros, 4 CPUs)

Tabela 12 – Métricas por CPU: cenário A

Escalonador	CPU	Útil	Troca	Ocioso	Útil %	Troca %	Ocioso %	Utilização %	Trocas
FCFS	0	8704	5525	110	60,70	38,53	0,77	99,23	5525
FCFS	1	8607	5524	208	60,03	38,52	1,45	98,55	5524
FCFS	2	8692	5464	183	60,62	38,11	1,28	98,72	5464
FCFS	3	8536	5481	322	59,53	38,22	2,25	97,75	5481
SJF	0	9062	2770	251	75,00	22,92	2,08	97,92	2770
SJF	1	8679	2857	547	71,83	23,64	4,53	95,47	2857
SJF	2	8414	2960	709	69,64	24,50	5,87	94,13	2960
SJF	3	8384	3037	662	69,39	25,13	5,48	94,52	3037
RR q=2	0	8663	7535	43	53,34	46,39	0,26	99,74	7535
RR q=2	1	8634	7556	51	53,16	46,52	0,31	99,69	7556
RR q=2	2	8650	7529	62	53,26	46,36	0,38	99,62	7529
RR q=2	3	8592	7541	108	52,90	46,43	0,66	99,34	7541
RR q=10	0	8693	5697	51	60,20	39,45	0,35	99,65	5697
RR q=10	1	8622	5721	98	59,71	39,62	0,68	99,32	5721
RR q=10	2	8584	5686	171	59,44	39,37	1,18	98,82	5686
RR q=10	3	8640	5645	156	59,83	39,09	1,08	98,92	5645
RR q=50	0	8719	5524	98	60,80	38,52	0,68	99,32	5524
RR q=50	1	8600	5522	219	59,97	38,50	1,53	98,47	5522
RR q=50	2	8652	5472	217	60,33	38,16	1,51	98,49	5472
RR q=50	3	8568	5482	291	59,74	38,23	2,03	97,97	5482

Cenário B (LRU, 256 quadros, 4 CPUs)

Tabela 13 – Métricas por CPU: cenário B

Escalonador	CPU	Útil	Troca	Ocioso	Útil %	Troca %	Ocioso %	Utilização %	Trocas
FCFS	0	8593	3037	278	72,16	25,50	2,33	97,67	3037
FCFS	1	8771	2881	256	73,66	24,19	2,15	97,85	2881
FCFS	2	8715	3051	142	73,19	25,62	1,19	98,81	3051
FCFS	3	8460	2997	451	71,04	25,17	3,79	96,21	2997
SJF	0	8698	2438	201	76,72	21,50	1,77	98,23	2438
SJF	1	8568	2124	645	75,58	18,74	5,69	94,31	2124
SJF	2	8562	2147	628	75,52	18,94	5,54	94,46	2147
SJF	3	8711	1638	988	76,84	14,45	8,71	91,29	1638
RR q=2	0	8657	6701	88	56,05	43,38	0,57	99,43	6701
RR q=2	1	8666	6654	126	56,11	43,08	0,82	99,18	6654
RR q=2	2	8613	6686	147	55,76	43,29	0,95	99,05	6686
RR q=2	3	8603	6667	176	55,70	43,16	1,14	98,86	6667
RR q=10	0	8819	3366	49	72,09	27,51	0,40	99,60	3366
RR q=10	1	8586	3385	263	70,18	27,67	2,15	97,85	3385
RR q=10	2	8603	3357	274	70,32	27,44	2,24	97,76	3357
RR q=10	3	8531	3351	352	69,73	27,39	2,88	97,12	3351
RR q=50	0	8792	3069	26	73,96	25,82	0,22	99,78	3069
RR q=50	1	8665	3022	200	72,89	25,42	1,68	98,32	3022
RR q=50	2	8553	3106	228	71,95	26,13	1,92	98,08	3106
RR q=50	3	8529	3068	290	71,75	25,81	2,44	97,56	3068